

8086 Assembly Programming

Hamim Ibne Nasim's notes

CSE341: Microprocessor

BRAC University

February 26, 2026

Contents

1	Basic Operations	3
1.1	Task: Input and Move Data	3
1.2	Task: Swap Two Numbers (Using 3 Registers)	3
1.3	Task: Swap Using ADD/SUB Only	4
1.4	Task: Add Two Numbers	4
1.5	Task: Subtract Two Numbers	5
1.6	Task: Arithmetic Operations with Variables	5
1.7	Task: Multiplication and Division	6
1.8	Task: Complex Arithmetic Expression	7
2	Input/Output Operations	9
2.1	Task: Character Input and Display	9
2.2	Task: Addition with User Input	9
2.3	Task: Uppercase to Lowercase Conversion	10
2.4	Task: Sum of Two Digits with Message	11
2.5	Task: Display Initials Vertically	13
2.6	Task: Hex to Decimal Conversion	14
2.7	Task: Display 10x10 Box of Asterisks	15
3	Conditional Statements	16
3.1	Task: Replace Negative with 5	16
3.2	Task: Display Character that Comes First	16
3.3	Task: Sign Indicator	17
3.4	Task: Odd/Even Display	17
3.5	Task: Display Uppercase Letter	18
3.6	Task: Even or Odd Number Check	19
3.7	Task: Vowel or Consonant	20
3.8	Task: Divisibility by 5 and 11	21
3.9	Task: Find Maximum of Three Numbers	22
3.10	Task: Day of Week Display	23
4	Loop Operations	25
4.1	Task: Display 80 Stars	25
4.2	Task: Sum of Arithmetic Series	25
4.3	Task: Multiplication by Repeated Addition	26
4.4	Task: Display Extended ASCII Characters	26
4.5	Task: Hex to Decimal Converter (Loop)	27
4.6	Task: Sum Numbers Divisible by Last Digit of ID	29
4.7	Task: Number Pattern (Nested Loop)	30
4.8	Task: Leap Year Checker	31
5	String Operations	33
5.1	Task: Palindrome Checker	33

6	Flag Operations	35
6.1	Task: Flag Status After Addition	35
6.2	Explanation: Overflow in Addition	35
7	Important Notes	36
7.1	Register Usage	36
7.2	Common DOS Interrupts (INT 21H)	36
7.3	ASCII Conversion	36
7.4	Loop Instructions	36
7.5	Jump Instructions	36
7.6	Data Definition	37
7.7	String Terminators	37

1 Basic Operations

1.1 Task: Input and Move Data

Take input in register AX and move it to BX.

```
1 .MODEL SMALL
2 .STACK 100H
3 .DATA
4 .CODE
5 MAIN PROC
6     ; Input number in AX
7     MOV AH, 1          ; DOS function for single character input
8     INT 21H
9     MOV BL, AL         ; Store first digit
10
11    ; Move to BX
12    MOV BH, 0
13
14
15    MOV AX, 4C00H      ; Exit
16    INT 21H
17 MAIN ENDP
18 END MAIN
```

Listing 1: Input and Move

1.2 Task: Swap Two Numbers (Using 3 Registers)

Swap values in AX and BX using temporary register CX.

```
1 .MODEL SMALL
2 .STACK 100H
3 .DATA
4 .CODE
5 MAIN PROC
6     MOV AX, 10         ; First number
7     MOV BX, 20         ; Second number
8
9     ; Swap using CX as temporary
10    MOV CX, AX          ; CX = AX
11    MOV AX, BX          ; AX = BX
12    MOV BX, CX          ; BX = CX
13
14    MOV AH, 4CH
15    INT 21H
16 MAIN ENDP
17 END MAIN
```

Listing 2: Swap with MOV

1.3 Task: Swap Using ADD/SUB Only

Swap two numbers without using a third register.

```

1  .MODEL  SMALL
2  .STACK  100H
3  .DATA
4  .CODE
5  MAIN  PROC
6      MOV  AX, 5          ; First number
7      MOV  BX, 10         ; Second number
8
9      ; Swap algorithm
10     ADD  AX, BX          ; AX = AX + BX
11     SUB  BX, AX          ; BX = BX - AX (now BX = original BX - (
                           ; original AX + original BX) = -original AX)
12     NEG  BX              ; BX = original AX
13     SUB  AX, BX          ; AX = (original AX + original BX) - original
                           ; AX = original BX
14
15     MOV  AH, 4CH
16     INT  21H
17 MAIN  ENDP
18 END  MAIN

```

Listing 3: Swap with ADD/SUB

1.4 Task: Add Two Numbers

```

1  .MODEL  SMALL
2  .STACK  100H
3  .DATA
4      NUM1  DB  5
5      NUM2  DB  3
6      RESULT DB  ?
7  .CODE
8  MAIN  PROC
9      MOV  AX, @DATA
10     MOV  DS, AX
11
12     MOV  AL, NUM1
13     ADD  AL, NUM2      ; AL = AL + NUM2
14     MOV  RESULT, AL
15
16     MOV  AH, 4CH
17     INT  21H
18 MAIN  ENDP
19 END  MAIN

```

Listing 4: Addition

1.5 Task: Subtract Two Numbers

Note: Subtracting larger from smaller gives negative result (2's complement).

```

1  .MODEL  SMALL
2  .STACK  100H
3  .DATA
4      NUM1  DB  10
5      NUM2  DB  7
6      RESULT  DB  ?
7  .CODE
8  MAIN  PROC
9      MOV  AX,  @DATA
10     MOV  DS,  AX
11
12     MOV  AL,  NUM1
13     SUB  AL,  NUM2      ;  AL = AL - NUM2
14     MOV  RESULT,  AL
15
16     ;  If  NUM2 > NUM1,  result  is  in  2's  complement  (negative)
17
18     MOV  AH,  4CH
19     INT  21H
20 MAIN  ENDP
21 END  MAIN

```

Listing 5: Subtraction

1.6 Task: Arithmetic Operations with Variables

```

1  .MODEL  SMALL
2  .STACK  100H
3  .DATA
4      A  DW  10
5      B  DW  5
6      C  DW  ?
7  .CODE
8  MAIN  PROC
9      MOV  AX,  @DATA
10     MOV  DS,  AX
11
12     ;  1.  A = B - A
13     MOV  AX,  B
14     SUB  AX,  A
15     MOV  A,  AX
16
17     ;  2.  A = -(A + 1)
18     MOV  AX,  A
19     ADD  AX,  1
20     NEG  AX
21     MOV  A,  AX

```

```

22
23 ; 3. C = A + (B + 1)
24 MOV AX, B
25 INC AX ; AX = B + 1
26 ADD AX, A
27 MOV C, AX
28
29 ; 4. A = B - (A - 1)
30 MOV AX, A
31 DEC AX ; AX = A - 1
32 MOV BX, B
33 SUB BX, AX
34 MOV A, BX
35
36 MOV AH, 4CH
37 INT 21H
38 MAIN ENDP
39 END MAIN

```

Listing 6: Variable Arithmetic

1.7 Task: Multiplication and Division

```

1 .MODEL SMALL
2 .STACK 100H
3 .DATA
4     X DW 6
5     Y DW 3
6     Z DW 2
7     RESULT DW ?
8 .CODE
9 MAIN PROC
10     MOV AX, @DATA
11     MOV DS, AX
12
13 ; 1. X * Y
14     MOV AX, X
15     MUL Y ; DX:AX = AX * Y
16     MOV RESULT, AX
17
18 ; 2. X / Y
19     MOV AX, X
20     MOV DX, 0 ; Clear DX for division
21     DIV Y ; AX = DX:AX / Y, DX = remainder
22     MOV RESULT, AX
23
24 ; 3. X * Y / Z
25     MOV AX, X
26     MUL Y ; DX:AX = X * Y
27     DIV Z ; AX = (X*Y) / Z
28     MOV RESULT, AX

```

```

29
30     MOV AH, 4CH
31     INT 21H
32 MAIN ENDP
33 END MAIN

```

Listing 7: MUL and DIV Operations

1.8 Task: Complex Arithmetic Expression

Calculate: $(1 + 2) * (3 - 1) / 5 + 3 + 2 - (1 * 2)$

```

1  .MODEL SMALL
2  .STACK 100H
3  .DATA
4      RESULT DW ?
5  .CODE
6  MAIN PROC
7      ; (1 + 2) = 3
8      MOV AX, 1
9      ADD AX, 2          ; AX = 3
10     MOV BX, AX          ; BX = 3
11
12     ; (3 - 1) = 2
13     MOV AX, 3
14     SUB AX, 1           ; AX = 2
15
16     ; 3 * 2 = 6
17     MUL BX              ; AX = 3 * 2 = 6
18
19     ; 6 / 5 = 1
20     MOV BX, 5
21     MOV DX, 0
22     DIV BX              ; AX = 6 / 5 = 1
23
24     ; + 3
25     ADD AX, 3           ; AX = 4
26
27     ; + 2
28     ADD AX, 2           ; AX = 6
29
30     ; (1 * 2) = 2
31     MOV BX, 1
32     MOV CX, 2
33     PUSH AX              ; Save current result
34     MOV AX, BX
35     MUL CX              ; AX = 2
36     MOV BX, AX
37     POP AX              ; Restore result
38
39     ; - 2
40     SUB AX, BX          ; AX = 6 - 2 = 4

```



```
41  
42     MOV RESULT , AX  
43  
44     MOV AH , 4CH  
45     INT 21H  
46 MAIN ENDP  
47 END MAIN
```

Listing 8: Complex Expression

2 Input/Output Operations

2.1 Task: Character Input and Display

```
1 .MODEL SMALL
2 .STACK 100H
3 .DATA
4     MSG DB 'Please insert a character: $'
5 .CODE
6 MAIN PROC
7     MOV AX, @DATA
8     MOV DS, AX
9
10    ; Display message
11    LEA DX, MSG
12    MOV AH, 9
13    INT 21H
14
15    ; Input character
16    MOV AH, 1
17    INT 21H
18    MOV BL, AL      ; Save character
19
20    ; Display character
21    MOV DL, BL
22    MOV AH, 2
23    INT 21H
24
25    MOV AH, 4CH
26    INT 21H
27 MAIN ENDP
28 END MAIN
```

Listing 9: Character I/O

2.2 Task: Addition with User Input

```
1 .MODEL SMALL
2 .STACK 100H
3 .DATA
4     MSG1 DB 'Enter first number: $'
5     MSG2 DB 0DH, 0AH, 'Enter second number: $'
6     MSG3 DB 0DH, 0AH, 'Sum is: $'
7 .CODE
8 MAIN PROC
9     MOV AX, @DATA
10    MOV DS, AX
11
12    ; Prompt for first number
13    LEA DX, MSG1
```

```

14     MOV AH, 9
15     INT 21H
16
17     ; Input first digit
18     MOV AH, 1
19     INT 21H
20     SUB AL, 30H      ; Convert ASCII to number
21     MOV BL, AL
22
23     ; Prompt for second number
24     LEA DX, MSG2
25     MOV AH, 9
26     INT 21H
27
28     ; Input second digit
29     MOV AH, 1
30     INT 21H
31     SUB AL, 30H
32
33     ; Add numbers
34     ADD BL, AL
35
36     ; Display result message
37     LEA DX, MSG3
38     MOV AH, 9
39     INT 21H
40
41     ; Display sum
42     MOV DL, BL
43     ADD DL, 30H      ; Convert to ASCII
44     MOV AH, 2
45     INT 21H
46
47     MOV AH, 4CH
48     INT 21H
49 MAIN ENDP
50 END MAIN

```

Listing 10: Interactive Addition

2.3 Task: Uppercase to Lowercase Conversion

```

1 .MODEL SMALL
2 .STACK 100H
3 .DATA
4     MSG DB 'Enter an uppercase letter: $'
5 .CODE
6 MAIN PROC
7     MOV AX, @DATA
8     MOV DS, AX
9

```

```

10      ; Display prompt
11      LEA DX, MSG
12      MOV AH, 9
13      INT 21H
14
15      ; Input uppercase letter
16      MOV AH, 1
17      INT 21H
18
19      ; Convert to lowercase (add 32 or 20H)
20      ADD AL, 20H
21
22      ; Move to next line
23      MOV DL, 0DH
24      MOV AH, 2
25      INT 21H
26      MOV DL, 0AH
27      INT 21H
28
29      ; Display lowercase
30      MOV DL, AL
31      INT 21H
32
33      MOV AH, 4CH
34      INT 21H
35 MAIN ENDP
36 END MAIN

```

Listing 11: Case Conversion

2.4 Task: Sum of Two Digits with Message

```

1  .MODEL SMALL
2  .STACK 100H
3  .DATA
4      MSG1 DB 0DH, 0AH, 'THE SUM OF $'
5      MSG2 DB ' AND $'
6      MSG3 DB ' IS $'
7  .CODE
8  MAIN PROC
9      MOV AX, @DATA
10     MOV DS, AX
11
12     ; Display ?
13     MOV DL, '?'
14     MOV AH, 2
15     INT 21H
16
17     ; Input first digit
18     MOV AH, 1
19     INT 21H

```

```
20      MOV BL, AL          ; Save first digit
21      SUB AL, 30H
22      MOV CL, AL          ; CL = first number
23
24      ; Input second digit
25      INT 21H
26      MOV BH, AL          ; Save second digit
27      SUB AL, 30H
28
29      ; Calculate sum
30      ADD CL, AL          ; CL = sum
31
32      ; Display message
33      LEA DX, MSG1
34      MOV AH, 9
35      INT 21H
36
37      ; Display first digit
38      MOV DL, BL
39      MOV AH, 2
40      INT 21H
41
42      ; Display "AND"
43      LEA DX, MSG2
44      MOV AH, 9
45      INT 21H
46
47      ; Display second digit
48      MOV DL, BH
49      MOV AH, 2
50      INT 21H
51
52      ; Display "IS"
53      LEA DX, MSG3
54      MOV AH, 9
55      INT 21H
56
57      ; Display sum
58      MOV DL, CL
59      ADD DL, 30H
60      MOV AH, 2
61      INT 21H
62
63      MOV AH, 4CH
64      INT 21H
65 MAIN ENDP
66 END MAIN
```

Listing 12: Digit Sum Display

2.5 Task: Display Initials Vertically

```
1 .MODEL SMALL
2 .STACK 100H
3 .DATA
4     MSG DB 'ENTER THREE INITIALS: $'
5 .CODE
6 MAIN PROC
7     MOV AX, @DATA
8     MOV DS, AX
9
10    ; Display prompt
11    LEA DX, MSG
12    MOV AH, 9
13    INT 21H
14
15    ; Input three characters
16    MOV AH, 1
17    INT 21H
18    MOV BL, AL      ; First initial
19
20    INT 21H
21    MOV BH, AL      ; Second initial
22
23    INT 21H
24    MOV CL, AL      ; Third initial
25
26    ; New line
27    MOV DL, 0DH
28    MOV AH, 2
29    INT 21H
30    MOV DL, 0AH
31    INT 21H
32
33    ; Display first initial
34    MOV DL, BL
35    INT 21H
36
37    ; New line
38    MOV DL, 0DH
39    INT 21H
40    MOV DL, 0AH
41    INT 21H
42
43    ; Display second initial
44    MOV DL, BH
45    MOV AH, 2
46    INT 21H
47
48    ; New line
49    MOV DL, 0DH
```

```

50     INT 21H
51     MOV DL, 0AH
52     INT 21H
53
54     ; Display third initial
55     MOV DL, CL
56     INT 21H
57
58     MOV AH, 4CH
59     INT 21H
60 MAIN ENDP
61 END MAIN

```

Listing 13: Vertical Display

2.6 Task: Hex to Decimal Conversion

```

1  .MODEL SMALL
2  .STACK 100H
3  .DATA
4      MSG1 DB 'ENTER A HEX DIGIT: $'
5      MSG2 DB 0DH, 0AH, 'IN DECIMAL IT IS $'
6  .CODE
7  MAIN PROC
8      MOV AX, @DATA
9      MOV DS, AX
10
11     ; Display prompt
12     LEA DX, MSG1
13     MOV AH, 9
14     INT 21H
15
16     ; Input hex digit (A-F)
17     MOV AH, 1
18     INT 21H
19
20     ; Convert to decimal
21     SUB AL, 37H      ; 'A' (65) - 37H = 10
22     MOV BL, AL
23
24     ; Display message
25     LEA DX, MSG2
26     MOV AH, 9
27     INT 21H
28
29     ; Display tens digit (1)
30     MOV DL, '1'
31     MOV AH, 2
32     INT 21H
33
34     ; Display units digit

```

```
35     MOV DL, BL
36     ADD DL, 30H
37     INT 21H
38
39     MOV AH, 4CH
40     INT 21H
41 MAIN ENDP
42 END MAIN
```

Listing 14: Hex to Decimal

2.7 Task: Display 10x10 Box of Asterisks

```
1  .MODEL SMALL
2  .STACK 100H
3  .DATA
4      LINE DB '*****', 0DH, 0AH, '$'
5  .CODE
6  MAIN PROC
7      MOV AX, @DATA
8      MOV DS, AX
9
10     MOV CX, 10      ; 10 rows
11
12 PRINT_ROW:
13     LEA DX, LINE
14     MOV AH, 9
15     INT 21H
16
17     LOOP PRINT_ROW
18
19     MOV AH, 4CH
20     INT 21H
21 MAIN ENDP
22 END MAIN
```

Listing 15: Asterisk Box

3 Conditional Statements

3.1 Task: Replace Negative with 5

```
1 .MODEL SMALL
2 .STACK 100H
3 .DATA
4 .CODE
5 MAIN PROC
6     MOV AX, -10      ; Test with negative number
7
8     CMP AX, 0
9     JGE POSITIVE     ; Jump if AX >= 0
10
11     ; AX is negative
12     MOV AX, 5
13
14 POSITIVE:
15     MOV AH, 4CH
16     INT 21H
17 MAIN ENDP
18 END MAIN
```

Listing 16: Negative Number Check

3.2 Task: Display Character that Comes First

```
1 .MODEL SMALL
2 .STACK 100H
3 .DATA
4 .CODE
5 MAIN PROC
6     MOV AL, 'B'      ; First character
7     MOV BL, 'A'      ; Second character
8
9     CMP AL, BL
10    JL DISPLAY_AL     ; If AL < BL
11
12    ; Display BL
13    MOV DL, BL
14    MOV AH, 2
15    INT 21H
16    JMP EXIT
17
18 DISPLAY_AL:
19    MOV DL, AL
20    MOV AH, 2
21    INT 21H
22
23 EXIT:
```

```

24      MOV AH, 4CH
25      INT 21H
26 MAIN ENDP
27 END MAIN

```

Listing 17: Character Comparison

3.3 Task: Sign Indicator

Put -1 in BX if AX is negative, 0 if zero, 1 if positive.

```

1  .MODEL SMALL
2  .STACK 100H
3  .DATA
4  .CODE
5  MAIN PROC
6      MOV AX, 10      ; Test value
7
8      CMP AX, 0
9      JL  NEGATIVE
10     JE  ZERO
11
12     ; Positive
13     MOV BX, 1
14     JMP EXIT
15
16 NEGATIVE:
17     MOV BX, -1
18     JMP EXIT
19
20 ZERO:
21     MOV BX, 0
22
23 EXIT:
24     MOV AH, 4CH
25     INT 21H
26 MAIN ENDP
27 END MAIN

```

Listing 18: Sign Detection

3.4 Task: Odd/Even Display

If AL contains 1 or 3, display "o"; if 2 or 4, display "e".

```

1  .MODEL SMALL
2  .STACK 100H
3  .DATA
4  .CODE
5  MAIN PROC
6      MOV AL, 3      ; Test value
7

```

```

8      CMP AL, 1
9      JE ODD
10     CMP AL, 3
11     JE ODD
12     CMP AL, 2
13     JE EVEN
14     CMP AL, 4
15     JE EVEN
16     JMP EXIT
17
18 ODD:
19     MOV DL, 'o'
20     MOV AH, 2
21     INT 21H
22     JMP EXIT
23
24 EVEN:
25     MOV DL, 'e'
26     MOV AH, 2
27     INT 21H
28
29 EXIT:
30     MOV AH, 4CH
31     INT 21H
32 MAIN ENDP
33 END MAIN

```

Listing 19: Odd/Even Character

3.5 Task: Display Uppercase Letter

```

1  .MODEL SMALL
2  .STACK 100H
3  .DATA
4      MSG DB 'Enter a character: $'
5  .CODE
6  MAIN PROC
7      MOV AX, @DATA
8      MOV DS, AX
9
10     ; Display prompt
11     LEA DX, MSG
12     MOV AH, 9
13     INT 21H
14
15     ; Input character
16     MOV AH, 1
17     INT 21H
18
19     ; Check if uppercase (A-Z: 65-90)
20     CMP AL, 'A'

```

```

21     JL  EXIT
22     CMP AL, 'Z'
23     JG  EXIT
24
25     ; Display uppercase letter
26     MOV DL, AL
27     MOV AH, 2
28     INT 21H
29
30 EXIT:
31     MOV AH, 4CH
32     INT 21H
33 MAIN ENDP
34 END  MAIN

```

Listing 20: Uppercase Filter

3.6 Task: Even or Odd Number Check

```

1  .MODEL  SMALL
2  .STACK  100H
3  .DATA
4      MSG_EVEN DB 'EVEN$'
5      MSG_ODD  DB 'ODD$'
6      NUM  DW 10
7  .CODE
8  MAIN PROC
9      MOV AX, @DATA
10     MOV DS, AX
11
12     MOV AX, NUM
13     MOV BL, 2
14     DIV BL           ; Divide by 2
15
16     CMP AH, 0        ; Check remainder
17     JE  EVEN
18
19     ; Odd
20     LEA DX, MSG_ODD
21     MOV AH, 9
22     INT 21H
23     JMP EXIT
24
25 EVEN:
26     LEA DX, MSG_EVEN
27     MOV AH, 9
28     INT 21H
29
30 EXIT:
31     MOV AH, 4CH
32     INT 21H

```

```
33 MAIN ENDP
34 END MAIN
```

Listing 21: Even/Odd Check

3.7 Task: Vowel or Consonant

```
1  .MODEL SMALL
2  .STACK 100H
3  .DATA
4      MSG1 DB 'Enter a letter: $'
5      MSG2 DB 0DH, 0AH, 'VOWEL$'
6      MSG3 DB 0DH, 0AH, 'CONSONANT$'
7  .CODE
8  MAIN PROC
9      MOV AX, @DATA
10     MOV DS, AX
11
12     ; Display prompt
13     LEA DX, MSG1
14     MOV AH, 9
15     INT 21H
16
17     ; Input character
18     MOV AH, 1
19     INT 21H
20
21     ; Check vowels: A, E, I, O, U (case-insensitive)
22     CMP AL, 'A'
23     JE VOWEL
24     CMP AL, 'a'
25     JE VOWEL
26     CMP AL, 'E'
27     JE VOWEL
28     CMP AL, 'e'
29     JE VOWEL
30     CMP AL, 'I'
31     JE VOWEL
32     CMP AL, 'i'
33     JE VOWEL
34     CMP AL, 'O'
35     JE VOWEL
36     CMP AL, 'o'
37     JE VOWEL
38     CMP AL, 'U'
39     JE VOWEL
40     CMP AL, 'u'
41     JE VOWEL
42
43     ; Consonant
44     LEA DX, MSG3
```

```

45     MOV AH, 9
46     INT 21H
47     JMP EXIT
48
49 VOWEL:
50     LEA DX, MSG2
51     MOV AH, 9
52     INT 21H
53
54 EXIT:
55     MOV AH, 4CH
56     INT 21H
57 MAIN ENDP
58 END MAIN

```

Listing 22: Vowel/Consonant Check

3.8 Task: Divisibility by 5 and 11

```

1  .MODEL SMALL
2  .STACK 100H
3  .DATA
4      MSG1 DB 'Divisible by both 5 and 11$'
5      MSG2 DB 'Not divisible$'
6      NUM DW 55
7  .CODE
8  MAIN PROC
9      MOV AX, @DATA
10     MOV DS, AX
11
12     ; Check divisibility by 5
13     MOV AX, NUM
14     MOV BL, 5
15     DIV BL
16     CMP AH, 0
17     JNE NOT_DIV
18
19     ; Check divisibility by 11
20     MOV AX, NUM
21     MOV BL, 11
22     DIV BL
23     CMP AH, 0
24     JNE NOT_DIV
25
26     ; Divisible by both
27     LEA DX, MSG1
28     MOV AH, 9
29     INT 21H
30     JMP EXIT
31
32 NOT_DIV:

```

```

33     LEA DX, MSG2
34     MOV AH, 9
35     INT 21H
36
37 EXIT:
38     MOV AH, 4CH
39     INT 21H
40 MAIN ENDP
41 END MAIN

```

Listing 23: Divisibility Check

3.9 Task: Find Maximum of Three Numbers

```

1  .MODEL SMALL
2  .STACK 100H
3  .DATA
4      NUM1 DW 2
5      NUM2 DW 3
6      NUM3 DW 4
7      MAX_VAL DW ?
8      MSG DB 'Maximum number is: $'
9  .CODE
10 MAIN PROC
11     MOV AX, @DATA
12     MOV DS, AX
13
14     MOV AX, NUM1
15     MOV BX, NUM2
16
17     ; Compare NUM1 and NUM2
18     CMP AX, BX
19     JG CHECK_THIRD
20     MOV AX, BX      ; BX is larger
21
22 CHECK_THIRD:
23     ; Compare with NUM3
24     MOV BX, NUM3
25     CMP AX, BX
26     JG FOUND_MAX
27     MOV AX, BX
28
29 FOUND_MAX:
30     MOV MAX_VAL, AX
31
32     ; Display message
33     LEA DX, MSG
34     MOV AH, 9
35     INT 21H
36
37     ; Display maximum (assuming single digit)

```

```

38     MOV DL, BYTE PTR MAX_VAL
39     ADD DL, 30H
40     MOV AH, 2
41     INT 21H
42
43     MOV AH, 4CH
44     INT 21H
45 MAIN ENDP
46 END MAIN

```

Listing 24: Maximum of Three

3.10 Task: Day of Week Display

```

1  .MODEL SMALL
2  .STACK 100H
3  .DATA
4      DAY DB 3           ; 1=Saturday, 2=Sunday, etc.
5      SAT DB 'Saturday$'
6      SUN DB 'Sunday$'
7      MON DB 'Monday$'
8      TUE DB 'Tuesday$'
9      WED DB 'Wednesday$'
10     THU DB 'Thursday$'
11     FRI DB 'Friday$'
12 .CODE
13 MAIN PROC
14     MOV AX, @DATA
15     MOV DS, AX
16
17     MOV AL, DAY
18
19     CMP AL, 1
20     JE SATURDAY
21     CMP AL, 2
22     JE SUNDAY
23     CMP AL, 3
24     JE MONDAY
25     CMP AL, 4
26     JE TUESDAY
27     CMP AL, 5
28     JE WEDNESDAY
29     CMP AL, 6
30     JE THURSDAY
31     CMP AL, 7
32     JE FRIDAY
33     JMP EXIT
34
35 SATURDAY:
36     LEA DX, SAT
37     JMP DISPLAY

```



```
38
39 SUNDAY :
40     LEA DX, SUN
41     JMP DISPLAY
42
43 MONDAY :
44     LEA DX, MON
45     JMP DISPLAY
46
47 TUESDAY :
48     LEA DX, TUE
49     JMP DISPLAY
50
51 WEDNESDAY :
52     LEA DX, WED
53     JMP DISPLAY
54
55 THURSDAY :
56     LEA DX, THU
57     JMP DISPLAY
58
59 FRIDAY :
60     LEA DX, FRI
61
62 DISPLAY :
63     MOV AH, 9
64     INT 21H
65
66 EXIT :
67     MOV AH, 4CH
68     INT 21H
69 MAIN ENDP
70 END MAIN
```

Listing 25: Day Name Display

4 Loop Operations

4.1 Task: Display 80 Stars

```

1  .MODEL  SMALL
2  .STACK  100H
3  .DATA
4  .CODE
5  MAIN PROC
6      MOV CX, 80          ; Loop counter
7
8  PRINT_STAR:
9      MOV DL, '*'
10     MOV AH, 2
11     INT 21H
12
13     LOOP PRINT_STAR
14
15     MOV AH, 4CH
16     INT 21H
17 MAIN ENDP
18 END MAIN

```

Listing 26: Simple Loop

4.2 Task: Sum of Arithmetic Series

Calculate sum: $1 + 4 + 7 + \dots + 148$

```

1  .MODEL  SMALL
2  .STACK  100H
3  .DATA
4      SUM DW 0
5  .CODE
6  MAIN PROC
7      MOV AX, @DATA
8      MOV DS, AX
9
10     MOV AX, 0          ; Sum accumulator
11     MOV BX, 1          ; Current number
12
13  SUM_LOOP:
14     ADD AX, BX          ; Add to sum
15     ADD BX, 3           ; Next number (increment by 3)
16
17     CMP BX, 148
18     JLE SUM_LOOP       ; Continue if BX <= 148
19
20     MOV SUM, AX
21
22     MOV AH, 4CH

```

```

23     INT 21H
24 MAIN ENDP
25 END MAIN

```

Listing 27: Arithmetic Series Sum

4.3 Task: Multiplication by Repeated Addition

```

1  .MODEL SMALL
2  .STACK 100H
3  .DATA
4      M DW 5
5      N DW 4
6      PRODUCT DW 0
7  .CODE
8  MAIN PROC
9      MOV AX, @DATA
10     MOV DS, AX
11
12     MOV AX, 0           ; Product = 0
13     MOV BX, M
14     MOV CX, N
15
16 MULT_LOOP:
17     ADD AX, BX          ; Product = Product + M
18     LOOP MULT_LOOP      ; Decrement CX and loop
19
20     MOV PRODUCT, AX
21
22     MOV AH, 4CH
23     INT 21H
24 MAIN ENDP
25 END MAIN

```

Listing 28: Multiplication Loop

4.4 Task: Display Extended ASCII Characters

```

1  .MODEL SMALL
2  .STACK 100H
3  .DATA
4  .CODE
5  MAIN PROC
6      MOV CX, 128        ; 128 characters (80h to FFh)
7      MOV DL, 80H        ; Start from 80h
8      MOV BL, 0          ; Counter for 10 per line
9
10 DISPLAY_CHAR:
11     MOV AH, 2
12     INT 21H

```

```

13
14     ; Space after character
15     PUSH DX
16     MOV DL, ' '
17     INT 21H
18     POP DX
19
20     INC DL
21     INC BL
22
23     ; Check if 10 characters printed
24     CMP BL, 10
25     JNE CONTINUE
26
27     ; New line
28     PUSH DX
29     MOV DL, 0DH
30     INT 21H
31     MOV DL, 0AH
32     INT 21H
33     POP DX
34
35     MOV BL, 0           ; Reset counter
36
37 CONTINUE:
38     LOOP DISPLAY_CHAR
39
40     MOV AH, 4CH
41     INT 21H
42 MAIN ENDP
43 END MAIN

```

Listing 29: Extended ASCII Display

4.5 Task: Hex to Decimal Converter (Loop)

```

1  .MODEL SMALL
2  .STACK 100H
3  .DATA
4      MSG1 DB 'ENTER A HEX DIGIT: $'
5      MSG2 DB 0DH, 0AH, 'IN DECIMAL IT IS $'
6      MSG3 DB 0DH, 0AH, 'DO YOU WANT TO DO IT AGAIN? $'
7  .CODE
8  MAIN PROC
9      MOV AX, @DATA
10     MOV DS, AX
11
12 START:
13     ; Display prompt
14     LEA DX, MSG1
15     MOV AH, 9

```

```
16      INT 21H
17
18      ; Input hex digit
19      MOV AH, 1
20      INT 21H
21      MOV BL, AL
22
23      ; Check if valid (0-9 or A-F)
24      CMP BL, '0'
25      JL INVALID
26      CMP BL, '9'
27      JLE VALID_DIGIT
28      CMP BL, 'A'
29      JL INVALID
30      CMP BL, 'F'
31      JG INVALID
32      JMP VALID_HEX
33
34 INVALID:
35      ; Error handling (skip for brevity)
36      JMP START
37
38 VALID_DIGIT:
39      SUB BL, 30H      ; Convert 0-9
40      JMP DISPLAY
41
42 VALID_HEX:
43      SUB BL, 37H      ; Convert A-F (A=10, etc.)
44
45 DISPLAY:
46      ; Display decimal message
47      LEA DX, MSG2
48      MOV AH, 9
49      INT 21H
50
51      ; Display tens digit if >= 10
52      CMP BL, 10
53      JL SINGLE_DIGIT
54
55      MOV DL, '1'
56      MOV AH, 2
57      INT 21H
58      SUB BL, 10
59
60 SINGLE_DIGIT:
61      MOV DL, BL
62      ADD DL, 30H
63      MOV AH, 2
64      INT 21H
65
66      ; Ask to continue
```

```

67     LEA DX, MSG3
68     MOV AH, 9
69     INT 21H
70
71     MOV AH, 1
72     INT 21H
73
74     CMP AL, 'y'
75     JE START
76     CMP AL, 'Y'
77     JE START
78
79     MOV AH, 4CH
80     INT 21H
81 MAIN ENDP
82 END MAIN

```

Listing 30: Hex Converter with Loop

4.6 Task: Sum Numbers Divisible by Last Digit of ID

```

1  .MODEL SMALL
2  .STACK 100H
3  .DATA
4      LAST_DIGIT DB 6      ; Last non-zero digit of ID
5      M DW 0              ; Sum of divisible numbers
6      N DW 0              ; Sum of non-divisible numbers
7  .CODE
8  MAIN PROC
9      MOV AX, @DATA
10     MOV DS, AX
11
12     MOV CX, 100           ; Loop from 1 to 100
13     MOV BX, 1            ; Current number
14
15 LOOP_START:
16     MOV AX, BX
17     MOV DL, LAST_DIGIT
18     DIV DL               ; Divide by last digit
19
20     CMP AH, 0            ; Check remainder
21     JE DIVISIBLE
22
23     ; Not divisible
24     ADD N, BX
25     JMP NEXT
26
27 DIVISIBLE:
28     ADD M, BX
29
30 NEXT:

```

```

31     INC BX
32     LOOP LOOP_START
33
34     MOV AH, 4CH
35     INT 21H
36 MAIN ENDP
37 END MAIN

```

Listing 31: Conditional Sum Loop

4.7 Task: Number Pattern (Nested Loop)

```

1  .MODEL SMALL
2  .STACK 100H
3  .DATA
4      INPUT DB 5           ; User input
5  .CODE
6  MAIN PROC
7      MOV AX, @DATA
8      MOV DS, AX
9
10     MOV CH, 0
11     MOV CL, INPUT        ; Outer loop counter
12     MOV BL, 1            ; Current row
13
14 OUTER_LOOP:
15     PUSH CX              ; Save outer counter
16     MOV CL, BL           ; Inner loop counter
17     MOV DL, '1'         ; Start from 1
18
19 INNER_LOOP:
20     MOV AH, 2
21     INT 21H
22
23     ; Space after number
24     PUSH DX
25     MOV DL, ' ',
26     INT 21H
27     POP DX
28
29     INC DL
30     DEC CL
31     JNZ INNER_LOOP
32
33     ; New line
34     PUSH DX
35     MOV DL, 0DH
36     MOV AH, 2
37     INT 21H
38     MOV DL, 0AH
39     INT 21H

```

```

40     POP DX
41
42     INC BL
43     POP CX           ; Restore outer counter
44     LOOP OUTER_LOOP
45
46     MOV AH, 4CH
47     INT 21H
48 MAIN ENDP
49 END MAIN

```

Listing 32: Pattern Display

4.8 Task: Leap Year Checker

```

1  .MODEL SMALL
2  .STACK 100H
3  .DATA
4      YEAR DW 1996
5      MSG_LEAP DB 'Leap year$'
6      MSG_NOT DB 'Not leap year$'
7  .CODE
8  MAIN PROC
9      MOV AX, @DATA
10     MOV DS, AX
11
12     ; Check divisibility by 400
13     MOV AX, YEAR
14     MOV BX, 400
15     MOV DX, 0
16     DIV BX
17     CMP DX, 0
18     JE LEAP
19
20     ; Check divisibility by 100
21     MOV AX, YEAR
22     MOV BX, 100
23     MOV DX, 0
24     DIV BX
25     CMP DX, 0
26     JE NOT_LEAP
27
28     ; Check divisibility by 4
29     MOV AX, YEAR
30     MOV BX, 4
31     MOV DX, 0
32     DIV BX
33     CMP DX, 0
34     JE LEAP
35
36 NOT_LEAP:

```



```
37      LEA DX, MSG_NOT
38      JMP DISPLAY
39
40 LEAP:
41      LEA DX, MSG_LEAP
42
43 DISPLAY:
44      MOV AH, 9
45      INT 21H
46
47      MOV AH, 4CH
48      INT 21H
49 MAIN ENDP
50 END MAIN
```

Listing 33: Leap Year Detection

5 String Operations

5.1 Task: Palindrome Checker

```

1  .MODEL  SMALL
2  .STACK  100H
3  .DATA
4      TEXT DB  'race car$'
5      CLEAN DB 30 DUP('$')
6      IS_PALINDROME DB 0
7      MSG1 DB  'Palindrome$'
8      MSG2 DB  'Not Palindrome$'
9  .CODE
10 MAIN PROC
11     MOV AX, @DATA
12     MOV DS, AX
13
14     ; Remove spaces - Loop 1
15     LEA SI, TEXT
16     LEA DI, CLEAN
17     MOV CX, 0                ; Clean length counter
18
19 REMOVE_SPACES:
20     MOV AL, [SI]
21     CMP AL, '$'
22     JE CHECK_PALINDROME
23
24     CMP AL, ' '              ; Skip space
25     JE SKIP
26
27     MOV [DI], AL
28     INC DI
29     INC CX
30
31 SKIP:
32     INC SI
33     JMP REMOVE_SPACES
34
35 CHECK_PALINDROME:
36     ; Check if palindrome - Loop 2
37     LEA SI, CLEAN            ; Left pointer
38     LEA DI, CLEAN
39     ADD DI, CX
40     DEC DI                    ; Right pointer
41
42     MOV BX, CX
43     SHR BX, 1                ; BX = length / 2
44
45 COMPARE_LOOP:
46     MOV AL, [SI]
47     MOV AH, [DI]

```

```
48     CMP AL, AH
49     JNE NOT_PALINDROME
50
51     INC SI
52     DEC DI
53     DEC BX
54     JNZ COMPARE_LOOP
55
56     ; Is palindrome
57     MOV IS_PALINDROME, 1
58     LEA DX, MSG1
59     JMP DISPLAY
60
61 NOT_PALINDROME:
62     MOV IS_PALINDROME, 0
63     LEA DX, MSG2
64
65 DISPLAY:
66     MOV AH, 9
67     INT 21H
68
69     MOV AH, 4CH
70     INT 21H
71 MAIN ENDP
72 END MAIN
```

Listing 34: Palindrome Detection

6 Flag Operations

6.1 Task: Flag Status After Addition

```
1  .MODEL  SMALL
2  .STACK  100H
3  .DATA
4  .CODE
5  MAIN PROC
6      ; ADD AL, BL where AL=80h, BL=80h
7      MOV AL, 80H
8      MOV BL, 80H
9
10     ADD AL, BL
11
12     ; Result: AL = 00H
13     ; CF = 1 (carry out)
14     ; OF = 1 (signed overflow: 128+128=-256 in signed)
15     ; ZF = 1 (result is zero)
16     ; SF = 0 (result is positive in signed)
17     ; PF = 1 (even parity)
18     ; AF = 1 (auxiliary carry)
19
20     ; To check flags, use PUSHF and analyze
21     PUSHF
22     POP AX
23     ; AX now contains FLAGS register
24
25     MOV AH, 4CH
26     INT 21H
27 MAIN ENDP
28 END MAIN
```

Listing 35: Flag Analysis

6.2 Explanation: Overflow in Addition

When adding two positive numbers (AX, BX both positive):

- Carry into MSB but no carry out = Signed overflow occurs
- Result appears negative (MSB=1) but both operands were positive

When adding two negative numbers (AX, BX both negative):

- Carry out of MSB but no carry in = Signed overflow occurs
- Result appears positive (MSB=0) but both operands were negative

7 Important Notes

7.1 Register Usage

- **AX, BX, CX, DX:** General purpose 16-bit registers
- **AL, BL, CL, DL:** Low byte of corresponding registers
- **AH, BH, CH, DH:** High byte of corresponding registers
- **SI, DI:** Source and destination index registers
- **SP, BP:** Stack and base pointers

7.2 Common DOS Interrupts (INT 21H)

- **AH=01H:** Single character input (result in AL)
- **AH=02H:** Single character output (character in DL)
- **AH=09H:** String output (address in DX, string ends with '\$')
- **AH=4CH:** Terminate program

7.3 ASCII Conversion

- **Number to ASCII:** ADD AL, 30H (or ADD AL, '0')
- **ASCII to Number:** SUB AL, 30H (or SUB AL, '0')
- **Uppercase to Lowercase:** ADD AL, 20H
- **Lowercase to Uppercase:** SUB AL, 20H

7.4 Loop Instructions

- **LOOP label:** Decrement CX and jump if CX $\neq$ 0
- **LOOPE/LOOPZ:** Loop while equal/zero
- **LOOPNE/LOOPNZ:** Loop while not equal/not zero

7.5 Jump Instructions

- **Unconditional:** JMP
- **Equal/Zero:** JE, JZ
- **Not Equal/Not Zero:** JNE, JNZ
- **Greater:** JG (signed), JA (unsigned)
- **Less:** JL (signed), JB (unsigned)
- **Greater or Equal:** JGE (signed), JAE (unsigned)
- **Less or Equal:** JLE (signed), JBE (unsigned)

7.6 Data Definition

- **DB**: Define Byte (8 bits)
- **DW**: Define Word (16 bits)
- **DD**: Define Double Word (32 bits)
- **DUP**: Duplicate values

7.7 String Terminators

- **'\$'**: DOS string terminator (for INT 21H, AH=09H)
- **0 or NULL**: C-style string terminator
- **0DH, 0AH**: Carriage return and line feed (new line)